DATA ANALYTICS PORTFOLIO PROJECT

Olist E-Commerce Analysis

From raw CSVs to business insights using Python, PostgreSQL and Power BI

Python | PostgreSQL | Power BI

Suman Rani

Project Report | October 2026

1. Project Overview

This project analyses the Olist Brazilian e-commerce marketplace dataset end to end. Nine raw CSV files were explored and cleaned in **Python (pandas)**, loaded into a **PostgreSQL** database, queried with SQL to answer ten business questions, and finally presented in an interactive **Power BI** dashboard.

Objectives

- Prepare a clean, analysis-ready database from nine related source tables.
- Answer key questions on revenue, shipping cost, delivery performance, payments, customers and product categories.
- Communicate the findings through a one-page dashboard that a business user can read at a glance.

Tools and Technologies

Tool	Used for
Python (pandas, NumPy)	Data inspection, missing-value treatment, duplicate checks
SQLAlchemy + psycopg2	Loading the cleaned DataFrames into PostgreSQL
PostgreSQL	Relational storage and 10 analytical queries (joins, aggregations, date functions)
Power BI	Interactive dashboard with KPIs, trends and category / geography breakdowns

Dataset

The dataset contains nine related tables covering the full order lifecycle:

Table	Contents
olist_customers	Customer IDs, city and state
olist_orders	Order status and purchase / approval / delivery timestamps
olist_order_items	Products, sellers, price and freight value per order
olist_order_payments	Payment type, installments and payment value
olist_order_reviews	Review score, title and comment
olist_products	Category and physical attributes of products
olist_sellers	Seller IDs and location
olist_geolocation	Zip-code level geographic coordinates
product_category_name_translation	Category names (Portuguese to English)

2. Methodology

The project follows a simple four-stage pipeline:

1 Explore	Check shape, columns, data types, nulls and duplicates for every table in Python.
2 Clean	Fill missing values, convert date columns and verify the result.
3 Load and Query	Store all nine tables in PostgreSQL and answer ten business questions with SQL.
4 Visualise	Build the Power BI dashboard on top of the analysed data and summarise insights.

Data Cleaning Summary

Table	Issue found	Treatment
olist_order_reviews	Missing review title and comment text	Filled with the placeholder "no review"
olist_orders	Missing approval, carrier and customer delivery timestamps	Converted to datetime and filled using interpolation
olist_products	Missing category and numeric product attributes	Category filled with the mode; name length, description length, photo count, weight and dimensions filled with the median
All nine tables	Unknown shape, types and duplicates	Checked with shape, info(), describe(), isnull().sum() and duplicate counts

3. Python: Exploration and Cleaning

Every table was loaded with pandas and run through the same checks: size, columns, summary statistics, missing values, data types and duplicates. The customers table, for example, was also checked for repeated *customer_id* and *customer_unique_id* values.

01_explore_customers.py (file path shortened)

```
import pandas as pd
import numpy as np

df = pd.read_csv("olist_customers_dataset.csv")

print(df.shape)
print(df.columns)
print(df.describe())
print(df.isnull().sum())
print(df.info())
print("Duplicate rows:", df.duplicated().sum())
print("Duplicate customer_id:", df["customer_id"].duplicated().sum())
print("Duplicate customer_unique_id:", df["customer_unique_id"].duplicated().sum())
```

Handling missing values

Reviews with no text were labelled explicitly. Order timestamps were converted to proper datetime types before filling gaps, and product attributes were filled with the mode (categorical) or median (numeric), which is robust to outliers.

02_clean_data.py (excerpt)

```
# Reviews: label empty comments
df4['review_comment_message'] = df4['review_comment_message'].fillna('no review')
df4['review_comment_title']   = df4['review_comment_title'].fillna('no review')

# Orders: convert to datetime, then fill gaps
df5['order_approved_at'] = pd.to_datetime(df5['order_approved_at'])
df5['order_approved_at'] = df5['order_approved_at'].interpolate()

# Products: mode for category, median for numeric columns
df6['product_category_name'] = df6['product_category_name'].fillna(
    df6['product_category_name'].mode()[0])
df6['product_weight_g'] = df6['product_weight_g'].fillna(df6['product_weight_g'].median())
df6['product_length_cm'] = df6['product_length_cm'].fillna(df6['product_length_cm'].median())
```

4. Python: Loading Data into PostgreSQL

After cleaning, all nine DataFrames were stored in a dictionary and written to the *olist_project* PostgreSQL database in a single loop using SQLAlchemy. This turned the CSV files into a proper relational database that can be joined and aggregated with SQL.

03_export_to_postgresql.py (credentials removed)

```
from sqlalchemy import create_engine

engine = create_engine(
    f"postgresql+psycopg2://{username}:{password}@{host}:{port}/{database}")

dataframes = {
    "olist_customers": df,          "olist_geolocation": df1,
    "olist_order_items": df2,       "olist_order_payments": df3,
    "olist_order_reviews": df4,     "olist_orders": df5,
    "olist_products": df6,          "olist_sellers": df7,
    "olist_product_category": df8,
}

for table_name, dataframe in dataframes.items():
    dataframe.to_sql(name=table_name, con=engine, if_exists="replace", index=False)
    print(f"{table_name} transferred successfully!")

engine.dispose()
```

Result: nine tables available in PostgreSQL, ready for analysis.

5. SQL Analysis

Ten business questions were answered with PostgreSQL queries. The queries use joins across up to three tables, aggregations, date functions, conditional counting and type casting.

#	Business question	Key SQL techniques
1	Which seller generates the highest revenue?	JOIN, SUM, GROUP BY, LIMIT
2	What is the total product revenue and how does it change monthly?	date_trunc, to_timestamp, to_char
3	What is the total freight cost and which category has the highest shipping cost?	JOIN, SUM, ORDER BY
4	What is the average delivery time and which orders were delivered late?	Timestamp arithmetic, AVG, date casting
5	Which payment methods are most popular and what is the average number of installments?	COUNT, AVG, GROUP BY
6	Which customer states and cities place the most orders?	COUNT(DISTINCT), JOIN
7	Which categories receive the highest average review score?	Three-table JOIN, AVG
8	Which states generate the highest total payment value?	Three-table JOIN, SUM
9	Which product categories generate the most revenue?	JOIN, SUM, GROUP BY
10	How do monthly orders, delivered orders and sales change over time?	FILTER clause, COUNT(DISTINCT), date_trunc

Questions 1 and 2: Top seller and monthly revenue

```sql
-- 1) Which sellers generate the highest revenue?

select s.seller_id ,round( sum(i.price)::numeric , 2) as revenue
from olist_sellers s
join olist_order_items i
on s.seller_id = i.seller_id
group by s.seller_id
order by revenue desc
limit 1;

-- 2) What is the total product revenue, and how does it change month by mo

--A) TOTAL REVENUE
select round(sum(price)::numeric,2)
from olist_order_items;

--B)REVENUE AS MONTH
select to_char(
                date_trunc(
                    'month',
                        to_timestamp
                (o.order_purchase_timestamp,'yyyy-mm-dd hh24:mi:ss') ),
                'mon yyyy'),
                round(sum(i.price)::numeric,2) as revenue
from olist_orders o
join olist_order_items i
on o.order_id=i.order_id
group by date_trunc(
    'month',
        to_timestamp(o.order_purchase_timestamp,'yyyy-mm-dd hh24:mi:ss')
)
order by date_trunc(
    'month',
        to_timestamp(o.order_purchase_timestamp,'yyyy-mm-dd hh24:mi:ss')
        ) ;
```

Figure 1: SQL queries

Q1 joins sellers to order items and sums price per seller to find the highest-revenue seller. **Q2** first calculates total revenue, then groups revenue by month by converting the purchase timestamp with *to_timestamp* and truncating it with *date_trunc*.

Questions 3 and 4A: Freight cost and average delivery time

```sql
-- 3) What is the total freight cost, and which product categories have the highest shipping costs

--A)  TOTAL FREIGHT VALUE
select round(sum(freight_value)::numeric, 2 )as total_freight_value
from olist_order_items;

--B) HIGHEST FREIGHT VALUE PRODUCT
select p.product_category_name,round(sum(i.freight_value)::numeric, 2) as shipping_value
from olist_products p
    join olist_order_items i
        on i.product_id = p.product_id
group by (p.product_category_name)
order by shipping_value desc
    limit 1;

-- 4) What is the average delivery time, and which orders were delivered late?

--A) AVG DELIVERY TIME
SELECT
       AVG(
            (
               order_delivered_customer_date -
                TO_TIMESTAMP(
            order_purchase_timestamp
            'YYYY-MM-DD HH24:MI:SS')
          )
        ) AS avg_delivery_days
FROM olist_orders
WHERE order_purchase_timestamp IS NOT NULL
  AND order_delivered_customer_date IS NOT NULL;
```

Figure 2: SQL queries

Q3 sums freight value overall and by product category to find the most expensive category to ship. **Q4A** subtracts the purchase timestamp from the customer delivery date and averages it to get the average delivery time.

Questions 4B and 5A: Late deliveries and payment methods

```sql
--  B) LATE DELIVERED OREDRS

SELECT
    order_id,
        TO_TIMESTAMP(
        order_estimated_delivery_date,
        'YYYY-MM-DD HH24:MI:SS'
    )::date AS estimated_delivery_date,
        order_delivered_customer_date::date,
                (order_delivered_customer_date::date
            -
            TO_TIMESTAMP(
                order_estimated_delivery_date,
                'YYYY-MM-DD HH24:MI:SS')::date
    ) AS days_late
FROM olist_orders
WHERE order_delivered_customer_date IS NOT NULL
  AND order_estimated_delivery_date IS NOT NULL
  AND order_delivered_customer_date::date >
      TO_TIMESTAMP(
        order_estimated_delivery_date,
        'YYYY-MM-DD HH24:MI:SS'
      )::date
ORDER BY days_late desc;

-- 5) Which payment methods are most popular, and what is the average number of installments?

--A) POPULAR PAYMENT METHODE
select payment_type,count(payment_value) as  no_of_payments
from olist_order_payments
group by payment_type
order by count(payment_value) desc;
```

Figure 3: SQL queries

Q4B lists every order delivered after its estimated delivery date and calculates how many days late it was, sorted from the worst delay down. **Q5A** counts payments by type to rank the most popular payment methods.

Questions 5B, 6 and 7: Installments, geography and review scores

```sql
--B) AVG NO. OF INSTALLMENT
select round(avg(payment_installments)::numeric,2) as avg_payment_installments
from olist_order_payments;

-- 6) Which customer cities or states have the highest number of orders?

--A) STATE WISE ORDERS
SELECT c.customer_state,count(DISTINCT o.order_id) as no_of_order
from olist_customers c
join olist_orders o
on c.customer_id = o.customer_id
group by c.customer_state
order by count(o.order_id) desc ;

--B) CITIES WISE ORDERS
SELECT c.customer_city,count(DISTINCT o.order_id) as no_of_order
from olist_customers c
join olist_orders o
on c.customer_id = o.customer_id
group by c.customer_city
order by count(o.order_id) desc ;

-- 7) Which product categories receive the highest average review scores?

SELECT p.product_category_name,round(avg(r.review_score)::numeric,2) as avg_review_score,count(*) as total_reviews
from olist_order_reviews r
join olist_order_items i
on r.order_id=i.order_id
join olist_products p
on p.product_id = i.product_id
group by p.product_category_name
order by avg(r.review_score) desc;
```

Figure 4: SQL queries

Q5B gives the average number of installments. **Q6** counts distinct orders by customer state and by city. **Q7** joins reviews, order items and products to rank categories by average review score alongside the number of reviews.

Questions 8, 9 and 10: Payments, revenue and monthly trend

```sql
-- 8) Which customer states generate the highest total payment value?

select c.customer_state,round(sum(p.payment_value)::numeric , 2) as total_payment_value
from olist_customers c
join olist_orders o
on o.customer_id= c.customer_id
join olist_order_payments p
on p.order_id = o.order_id
group by c.customer_state
order by total_payment_value desc;

-- 9) Which product categories generate the most revenue?

select p.product_category_name,round(sum(i.price)::numeric,2) as revenue
from olist_order_items i
join olist_products p
on i.product_id = p.product_id
group by p.product_category_name
order by  revenue desc;

-- 10) How do monthly orders, delivered orders, and product sales change over time?

select date_trunc('month',to_timestamp(o.order_purchase_timestamp,'yyyy-mm-dd hh24:mi:ss')) as month,
        count(distinct o.order_id) as total_orders,
        count(distinct o.order_id ) filter(
    where o.order_status = 'delivered')as delivered_order,
round(sum(i.price)::numeric,2) as product_sales
from olist_order_items i
join olist_orders o
on i.order_id = o.order_id
group by date_trunc('month',to_timestamp(o.order_purchase_timestamp,'yyyy-mm-dd hh24:mi:ss'))
order by month;
```

Figure 5: SQL queries

Q8 totals payment value by customer state. **Q9** ranks product categories by revenue. **Q10** combines total orders, delivered orders (using a *FILTER* clause) and product sales per month in a single result.

6. Power BI Dashboard

The Olist E-Commerce Dashboard summarises overall marketplace performance on a single page, with an order-month slicer and navigation for Sales & Revenue, Customers, Products, Delivery and Payments.

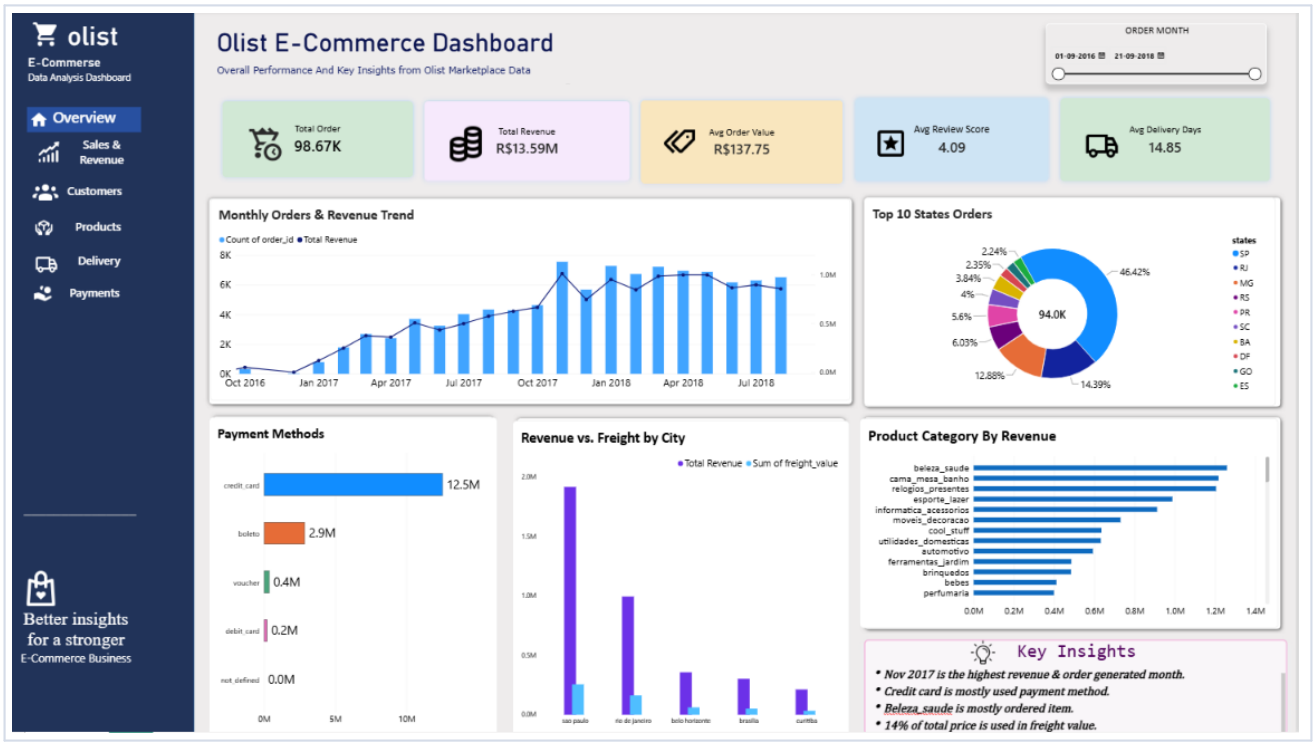


Figure 6: Olist E-Commerce Dashboard (Overview page)

98.67K	R$13.59M	R$137.75	4.09	14.85
Total Orders	Total Revenue	Avg Order Value	Avg Review Score	Avg Delivery Days

Visuals on the dashboard

- **KPI cards:** total orders, total revenue, average order value, average review score and average delivery days.
- **Monthly Orders & Revenue Trend:** combined bar and line chart from October 2016 to September 2018.
- **Top 10 States Orders:** donut chart showing each state's share of orders.
- **Payment Methods:** payment value by type.
- **Revenue vs. Freight by City:** revenue compared with freight cost for the leading cities.
- **Product Category by Revenue:** ranked bar chart of the top categories.

7. Key Findings

The main insights from the dashboard are:

Area	Finding
Growth	Orders and revenue rose steadily from late 2016, peaked in **November 2017** (the highest month for both), and stayed at a high level through 2018.
Geography	**São Paulo (SP)** accounts for 46.42% of orders among the top 10 states, followed by Rio de Janeiro (14.39%) and Minas Gerais (12.88%). Together these three make up nearly 74% of those orders.
Payments	**Credit card** is by far the most used method (R$12.5M), followed by boleto (R$2.9M), voucher (R$0.4M) and debit card (R$0.2M).
Products	**beleza_saude** (health and beauty) is the top revenue category, followed by cama_mesa_banho (bed, bath and table) and relogios_presentes (watches and gifts).
Shipping	About **14% of the total price** paid is freight value, so shipping is a significant part of customer cost.
Customer experience	The average review score is **4.09** and the average delivery time is **14.85 days**.
Cities	São Paulo and Rio de Janeiro lead in revenue and also carry the highest freight totals in the revenue vs. freight comparison.

8. Conclusion and Future Scope

This project took raw marketplace data through a complete analytics workflow: cleaning in Python, structured storage and analysis in PostgreSQL, and reporting in Power BI. The results show a marketplace that grew quickly, depends heavily on a few southeastern states, relies mainly on credit-card payments, and spends a noticeable share of order value on freight.

Possible next steps

- Link delivery delays to review scores to measure how late deliveries affect customer satisfaction.
- Analyse seller performance beyond the single top seller.
- Use *customer_unique_id* to study repeat purchases and customer retention.
- Add Python visualisations and a simple sales forecast for upcoming months.

Skills demonstrated

Python (pandas, NumPy, SQLAlchemy) | PostgreSQL | Data cleaning | Joins and aggregations | Date and time functions | Power BI dashboards | Data storytelling